⍰ React Native Fullstack

Formulário de Cadastro + Backend + MongoDB Atlas

Usando GitHub Codespace + Expo Snack - Professor Bruno Zolotareff dos Santos

Visão Geral

Este guia mostra como criar um formulário de cadastro completo (nome, e-mail, telefone, login e senha) em React Native, integrado a um backend Node.js hospedado no GitHub Codespace e um banco de dados MongoDB Atlas na nuvem — tudo gratuito, sem instalar nada.

O QUE VAMOS CONSTRUIR

1. App.js no Expo Snack (online) — formulário de cadastro no celular
2. Backend Node.js + Express no GitHub Codespace
3. Banco de dados MongoDB Atlas (nuvem gratuita)

Passo 1 — Criando o Repositório e abrindo o Codespace

1. Acesse github.com e faça login (crie uma conta gratuita se não tiver)
2. Clique em "New repository" → nome: cadastro-backend → Create
3. Na página do repositório → botão verde "Code" → aba "Codespaces" → "Create codespace on main"
4. Aguarde ~30 segundos. Um VS Code completo vai abrir no navegador.

> ⍰ O Codespace já tem Node.js, npm e terminal integrado. Não precisa instalar nada no seu computador.

Passo 2 — Criando a estrutura no terminal

No terminal do Codespace (Ctrl + ` para abrir), execute esses comandos um por vez:

```
Terminal do Codespace
mkdir backend && cd backend
npm init -y
npm install express mongoose cors bcryptjs dotenv
mkdir models
```

Passo 3 — Arquivo do Model (Usuario.js)

No painel esquerdo do Codespace, clique com botão direito em backend/models → New File → Usuario.js

```
 backend/models/Usuario.js
const mongoose = require('mongoose');

const UsuarioSchema = new mongoose.Schema({
  nome:     { type: String, required: true },
  email:    { type: String, required: true, unique: true },
  telefone: { type: String, default: '' },
  login:    { type: String, required: true, unique: true },
  senha:    { type: String, required: true },
}, { timestamps: true });

module.exports = mongoose.model('Usuario', UsuarioSchema);
```

Passo 4 — Arquivo principal (server.js)

Crie o arquivo backend/server.js:

```
 backend/server.js
require('dotenv').config();
const express  = require('express');
const mongoose = require('mongoose');
const cors     = require('cors');
const bcrypt   = require('bcryptjs');
const Usuario  = require('./models/Usuario');
```

```
const app = express();
app.use(cors());
app.use(express.json());

mongoose.connect(process.env.MONGO_URL)
  .then(() => console.log('MongoDB Atlas conectado!'))
  .catch(err => console.log('Erro:', err));

app.get('/', (req, res) => res.send('API funcionando!'));

app.post('/usuarios', async (req, res) => {
  try {
    const { nome, email, telefone, login, senha } = req.body;

    const existe = await Usuario.findOne({
      $or: [{ email }, { login }]
    });
    if (existe) {
      return res.status(400).json({ mensagem: 'E-mail ou login já cadastrado.' });
    }

    const senhaCripto = await bcrypt.hash(senha, 10);
    const usuario = await Usuario.create({
      nome, email, telefone, login, senha: senhaCripto
    });

    const { senha: _, ...publico } = usuario.toObject();
    res.status(201).json(publico);

  } catch (e) {
    res.status(500).json({ mensagem: 'Erro interno.' });
  }
});

app.get('/usuarios', async (req, res) => {
  const lista = await Usuario.find().select('-senha');
  res.json(lista);
});

app.listen(3000, () => console.log('Servidor rodando na porta 3000'));
```

> 🔒 A senha nunca é salva em texto puro. O bcryptjs criptografa antes de salvar no banco — isso é obrigatório em qualquer sistema real.

Passo 5 — MongoDB Atlas (banco de dados)

1. Acesse mongodb.com/cloud/atlas e crie uma conta gratuita
2. Clique em "Build a Database" → Free (M0) → Create
3. Crie um usuário com nome e senha — anote esses dados!
4. Em "Network Access" → Add IP Address → adicione 0.0.0.0/0
5. No cluster → Connect → Connect your application → copie a URL

A URL VAI PARECER ASSIM

```
String de conexão MongoDB Atlas
mongodb+srv://SEU_USUARIO:SUA_SENHA@cluster0.xxxxx.mongodb.net/cadastros
```

Passo 6 — Variável de ambiente (.env)

Crie o arquivo backend/.env no Codespace:

```
 backend/.env
MONGO_URL=mongodb+srv://SEU_USUARIO:SUA_SENHA@cluster0.xxxxx.mongodb.net/cadastros
```

Crie também o backend/.gitignore para não enviar a senha ao GitHub:

```
 backend/.gitignore
node_modules
.env
```

> ⚠ Nunca coloque a senha direto no código. O arquivo .env fica só no seu Codespace e nunca vai para o GitHub.

Passo 7 — Rodando o servidor no Codespace

No terminal do Codespace:

```
Terminal do Codespace
cd backend
node server.js
```

Você verá no terminal:

```
Saída esperada
✅ MongoDB Atlas conectado!
🚀 Servidor rodando na porta 3000
```

PEGANDO A URL PÚBLICA

1. Uma notificação vai aparecer: "Your app on port 3000 is available"
2. Clique na aba "Ports" na barra inferior do Codespace
3. Clique com botão direito na porta 3000 → Port Visibility → Public
4. Clique no ícone de globo para copiar a URL. Ex: https://xyz-3000.app.github.dev

> ⚠️ Se a porta ficar como Private, o Expo Snack não conseguirá se conectar. Certifique-se de deixar como Public.

Passo 8 — App.js no Expo Snack

Acesse snack.expo.dev, abra o App.js e cole o código abaixo. Substitua a URL pela sua URL real do Codespace.

```
📱 App.js — Expo Snack (código completo)
import { useState } from 'react';
import {
  View, Text, TextInput, TouchableOpacity,
  ScrollView, StyleSheet, Alert, ActivityIndicator
} from 'react-native';

const API = 'https://xyz-3000.app.github.dev'; // sua URL do Codespace

export default function App() {
  const [form, setForm] = useState({
    nome: '', email: '', telefone: '', login: '', senha: ''
  });
  const [loading, setLoading] = useState(false);

  function atualizar(campo, valor) {
    setForm({ ...form, [campo]: valor });
  }

  async function cadastrar() {
    if (!form.nome || !form.email || !form.login || !form.senha) {
      Alert.alert('Erro', 'Preencha todos os campos obrigatórios.');
      return;
```

```
    }
    setLoading(true);
    try {
      const res = await fetch(API + '/usuarios', {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify(form),
      });
      const dados = await res.json();
      if (res.ok) {
        Alert.alert('Sucesso!', 'Usuário cadastrado!');
        setForm({ nome:'', email:'', telefone:'', login:'', senha:'' });
      } else {
        Alert.alert('Erro', dados.mensagem);
      }
    } catch (e) {
      Alert.alert('Erro', 'Não foi possível conectar.');
    } finally {
      setLoading(false);
    }
  }

  return (
    <ScrollView contentContainerStyle={s.container}>
      <Text style={s.titulo}>Criar conta</Text>
      <Text style={s.sub}>Preencha seus dados abaixo</Text>

      <Text style={s.label}>Nome completo *</Text>
      <TextInput style={s.input} placeholder="Ex: João Silva"
        value={form.nome} onChangeText={v => atualizar("nome", v)} />

      <Text style={s.label}>E-mail *</Text>
      <TextInput style={s.input} placeholder="joao@email.com"
        keyboardType="email-address" autoCapitalize="none"
        value={form.email} onChangeText={v => atualizar("email", v)} />

      <Text style={s.label}>Telefone</Text>
      <TextInput style={s.input} placeholder="(11) 99999-9999"
        keyboardType="phone-pad"
        value={form.telefone} onChangeText={v => atualizar("telefone", v)} />

      <Text style={s.label}>Login *</Text>
      <TextInput style={s.input} placeholder="nome_usuario"
        autoCapitalize="none"
        value={form.login} onChangeText={v => atualizar("login", v)} />
```

```
      <Text style={s.label}>Senha *</Text>
      <TextInput style={s.input} placeholder="Mínimo 6 caracteres"
        secureTextEntry
        value={form.senha} onChangeText={v => atualizar("senha", v)} />

      <TouchableOpacity style={[s.btn, loading && s.btnOff]}
        onPress={cadastrar} disabled={loading}>
        {loading
          ? <ActivityIndicator color="#fff" />
          : <Text style={s.btnTxt}>Cadastrar</Text>}
      </TouchableOpacity>
    </ScrollView>
  );
}

const s = StyleSheet.create({
  container: { padding:24, paddingTop:60, backgroundColor:'#f5f5f5', flexGrow:1 },
  titulo:    { fontSize:28, fontWeight:'bold', color:'#1a1a1a' },
  sub:       { fontSize:15, color:'#888', marginBottom:24, marginTop:4 },
  label:     { fontSize:13, fontWeight:'600', color:'#444', marginBottom:6,
marginTop:14 },
  input:     { backgroundColor:'#fff', borderWidth:1, borderColor:'#ddd',
               borderRadius:10, padding:12, fontSize:15 },
  btn:       { backgroundColor:'#1D9E75', borderRadius:10, padding:15,
               alignItems:'center', marginTop:28 },
  btnOff:    { backgroundColor:'#aaa' },
  btnTxt:    { color:'#fff', fontSize:16, fontWeight:'bold' },
});
```

> 💾 Salve o arquivo no Snack — o app atualiza automaticamente no Expo Go do celular.

Como funciona tudo junto

1. Expo Snack (App.js) → usuário preenche o formulário e clica em Cadastrar
2. App envia um POST com os dados em JSON para a URL do Codespace
3. Codespace (server.js) recebe os dados e verifica se e-mail/login já existem
4. A senha é criptografada com bcrypt antes de salvar
5. Os dados são salvos no MongoDB Atlas na nuvem
6. O app exibe "Usuário cadastrado!" e limpa o formulário

Próximos desafios

1. Tela de login — Rota POST /login que verifica senha com bcrypt.compare()
2. Validação de e-mail — checar formato antes de enviar
3. Listar usuários — nova tela com GET /usuarios em uma FlatList
4. React Navigation — navegar entre tela de cadastro e tela de login
5. Deploy permanente — subir para o Render.com para ficar no ar sem o Codespace aberto

> ⚠ O servidor fica online apenas enquanto o Codespace está aberto. Para uso em produção, faça o deploy no Render.com.